

TP 17 — Le match des tris

Semaine 17 ■ Fichiers dans NSI/TP17/ ■ 2 h

Objectifs

- Implémenter le tri par insertion, valider avec le banc de tests du TP 16
- Mesurer meilleur cas / pire cas et confronter à la sélection
- Choisir le bon tri selon la forme des données

Partie A — Implémentation — `tris.py`

(35 min)

1. Recopier `tri_selection` (TP 16) et implémenter `tri_insertion` (cours).
2. Faire passer à `tri_insertion` **exactement le même** jeu de tests + le test de masse (arbitre **sorted**). Un banc de tests se réutilise !
3. Piège vérifié : trier `[5, 1]` pas à pas au débogueur ; observer `j`, `valeur`, et l'arrêt du **while** à `j == 0`.

Partie B — Meilleur et pire cas — `match.py`

(45 min)

Écrire `tri_insertion_compte(t)` (renvoie le nombre de comparaisons du **while**, y compris celle qui le fait échouer), puis remplir ce tableau pour $n = 1000$:

tableau de départ	sélection	insertion
aléatoire		
déjà trié		
ordre inverse		
presque trié (10 échanges aléatoires)		

Générer les entrées : `list(range(n))`, `list(range(n, 0, -1))`, `random.shuffle`, et pour « presque trié » : partir du trié et échanger 10 paires au hasard.

Questions (en commentaire) :

1. Où l'insertion écrase-t-elle la sélection ? Où sont-elles à égalité ?
2. Pour le classement du tournoi (exercice 4), chiffrer le gain.

Partie C — La courbe ★ — `courbe.py`

(25 min)

Chronométrer les deux tris sur tableaux aléatoires pour $n = 500, 1000, 2000, 4000$ et afficher un « graphique console » : une ligne par mesure, un caractère # par tranche de 10 ms.

```
sel  n=4000  ##### (0.61 s)
ins  n=4000  ##### (0.45 s)
```

Partie D — Défi ★★★

(15 min)

Le tri par insertion **dichotomique** : dans la zone triée, la place de `valeur` peut se trouver par dichotomie (semaine 15!) au lieu de comparer en glissant. Implémenter `tri_insertion_dicho` ; le nombre de **comparaisons** chute-t-il ? Et le temps total ? Pourquoi ?